



HTML & CSS



Obiettivo della lezione

Oggi introduciamo HTML e CSS.

Vedremo le basi e come integrarle nel nostro progetto

UTILITY:

- **MDN** (Mozilla Developer Network) è la principale documentazione di riferimento per le tecnologie web, incluso **HTML**.



Git checkpoint

Prima di scrivere codice, controlliamo lo stato della repository.

```
git pull  
git status  
git log --oneline
```

Controllo minimo:

- il README esiste?
- la cartella `docs/` esiste?
- ci sono file modificati non salvati?
- la repository è sincronizzata con GitHub?
- ci sono commit sensati?

Se tutto è a posto, creiamo i file della giornata:

```
touch index.html  
touch style.css
```

Cos'è HTML

HTML significa **HyperText Markup Language**.

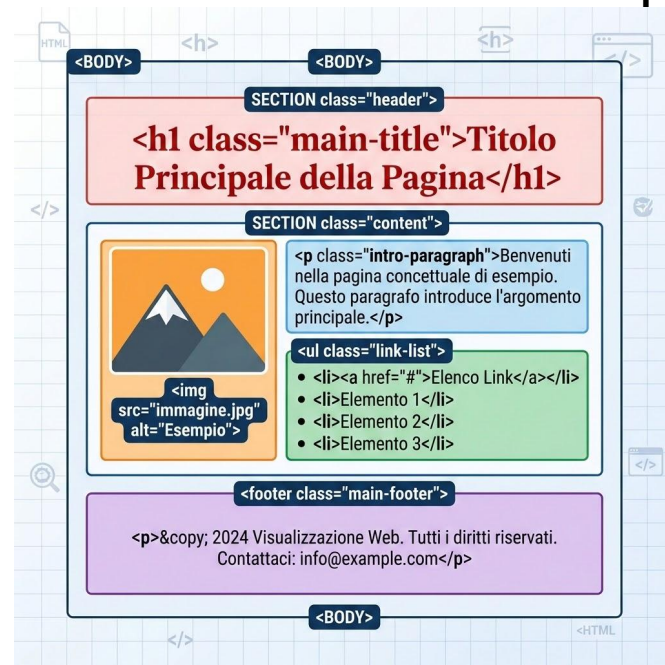
HTML è il linguaggio che definisce la struttura e il significato del contenuto di una pagina web.

Non serve principalmente a “fare bello” il sito.

Serve a dire al browser:

- questo è un titolo;
- questo è un paragrafo;
- questa è una lista;
- questo è un link;
- questa è un'immagine;
- questa è una sezione;
- questa è una tabella;
- questo è un form.

MDN definisce HTML come il blocco base del Web: serve a descrivere il significato e la struttura del contenuto; CSS e JavaScript si occupano rispettivamente di presentazione e comportamento.



HTML, CSS e JavaScript: differenza

Una pagina web può essere pensata in tre livelli.

Esempio:

```
<h1>Catalogo Film</h1>
```

```
<p>Una pagina per consultare una lista di film.</p>
```

```
h1 {  
  color: #222;  
  font-size: 36px;  
}
```

```
console.log("Pagina caricata");
```

Oggi ci concentriamo su HTML e CSS.

Tecnologia	Domanda	Esempio
HTML	Che cosa c'è nella pagina?	titolo, testo, lista, immagine
CSS	Come appare?	colori, margini, font, layout
JavaScript	Che cosa fa?	click, filtro, ricerca, interazione

Struttura minima di una pagina HTML

Ogni pagina HTML dovrebbe avere una struttura base.

Significato:

- `<!DOCTYPE html>` dice al browser che stiamo usando HTML moderno;
- `<html lang="it">` indica la lingua principale della pagina;
- `<head>` contiene informazioni sulla pagina;
- `<meta charset="UTF-8">` gestisce correttamente caratteri e accenti;
- `<title>` è il titolo mostrato nella scheda del browser;
- `<body>` contiene ciò che l'utente vede.

```
<!DOCTYPE html>
<html lang="it">
<head>
  <meta charset="UTF-8">
  <title>Nome progetto</title>
</head>
<body>
  <h1>Nome progetto</h1>
  <p>Descrizione del progetto.</p>
</body>
</html>
```

Tag, elementi e attributi

Un elemento HTML di solito ha:

- tag di apertura;
- contenuto;
- tag di chiusura.

Esempio:

```
<p>Questo è un paragrafo.</p>
```

Alcuni elementi hanno attributi.

Esempio:

```
<a href="https://github.com">Vai a GitHub</a>
```

Qui:

- **a** è il tag del link;
- **href** è l'attributo;
- il valore dell'attributo è l'indirizzo;
- "Vai a GitHub" è il testo visibile.

Altro esempio:

```

```

Qui:

- **src** indica dove si trova l'immagine;
- **alt** descrive l'immagine.

L'attributo **alt** è importante per accessibilità e fallback quando l'immagine non viene caricata.



Tag, elementi e attributi

Mini Prova:

Correggere questo codice:

```
<p>Benvenuto nella pagina  
<a>Documentazione</a>  

```




Tag, elementi e attributi

Mini Prova:

Soluzione

```
<p>Benvenuto nella pagina.</p>
```

```
<a href="docs/guida-utente.md">Documentazione</a>
```

```

```



Titoli e paragrafi

I titoli servono a costruire la struttura logica della pagina

```
<h1>Titolo principale</h1>
```

```
<h2>Sezione principale</h2>
```

```
<h3>Sottosezione</h3>
```

. Regola pratica:

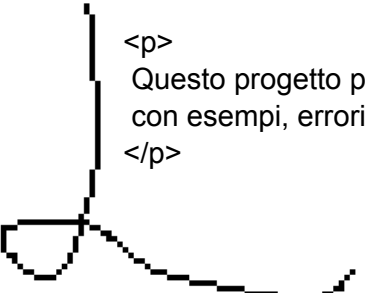
- usare un solo **h1** principale;
- usare **h2** per le sezioni;
- usare **h3** per sottosezioni;
- non scegliere i titoli solo perché “sono più grandi”.

I paragrafi si scrivono con **<p>**:

```
<p>
```

Questo progetto permette di consultare una lista di comandi Git con esempi, errori comuni e soluzioni.

```
</p>
```



Da Markdown a HTML

Guida Git

Obiettivo

Questa guida aiuta a imparare Git.



<h1>Guida Git</h1>

<h2>Obiettivo</h2>

<p>Questa guida aiuta a imparare Git.</p>

CODICE MARKDOWN

```
1 ---
2 title: Esempio
  Markdown
3 ---
4
5 # Primo Livello
6 ## Secondo Livello
7 ## Secondo Livello
8
9 Questo è un
  paragrafo di testo
  standard.
```

STRUTTURA HTML

```
1 <html>
2   <head>
3     <title>Esempio
  HTML</title>
4   </head>
5   <body>
6
7     <h1>Primo
  Livello</h1>
8     <h2>Secondo
  Livello</h2>
9     <p>Questo è un
  paragrafo di testo
  standard.</p>
10
11   </body>
12 </html>
```



Liste

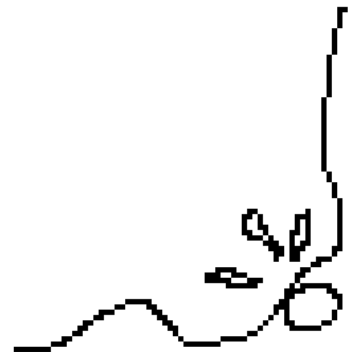
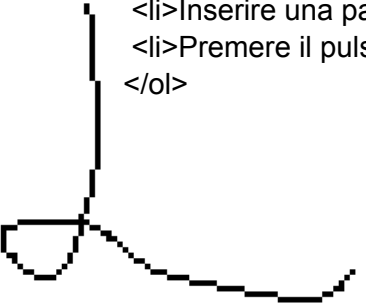
Le liste servono a organizzare informazioni ripetute.

Lista puntata:

```
<ul>  
<li>Ricerca dati</li>  
<li>Filtro per categoria</li>  
<li>Visualizzazione dettagli</li>  
</ul>
```

Lista numerata:

```
<ol>  
<li>Aprire la pagina.</li>  
<li>Inserire una parola chiave.</li>  
<li>Premere il pulsante Cerca.</li>  
</ol>
```





Esercizio - Liste

Convertire questo Markdown in HTML:

Funzionalità

- Ricerca
- Filtro
- Visualizzazione dettagli



Esercizio - Liste

Soluzione

```
<section id="funzionalita">  
<h2>Funzionalità</h2>  
<ul>  
  <li>Ricerca</li>  
  <li>Filtro</li>  
  <li>Visualizzazione dettagli</li>  
</ul>  
</section>
```



Link

I link collegano la pagina ad altre risorse.

Sintassi:

```
<a href="percorso-o-url">Testo visibile</a>
```

Esempi:

```
<a href="README.md">Leggi il README</a>
```

```
<a href="docs/installazione.md">Guida installazione</a>
```

```
<a href="https://github.com">Vai a GitHub</a>
```

Regola importante:

Il testo del link deve essere chiaro.



Scarso:

```
<a href="docs/guida-utente.md">clicca  
qui</a>
```

Meglio:

```
<a href="docs/guida-utente.md">Leggi la  
guida utente</a>
```



Immagini

Le immagini si inseriscono con `<img>`.

```

```

Attributi principali:

- `src`: percorso dell'immagine;
- `alt`: descrizione testuale;
- `width`: larghezza;
- `height`: altezza.

Buon testo alternativo:

```

```

Testo alternativo scarso:

```

```


Immagini - Esercizio

Inserire uno screenshot del progetto dentro `index.html`.

```
<section id="screenshot">
  <h2>Screenshot</h2>
  <p>Questa schermata mostra la pagina principale del progetto.</p>
  
</section>
```

HTML semantico

HTML semantico significa usare tag che descrivono il ruolo del contenuto (sono essenzialmente dei tag `<div>`).

`<header>`Intestazione della pagina`</header>`

`<nav>`Menu di navigazione`</nav>`

`<main>`Contenuto principale`</main>`

`<section>`Sezione tematica`</section>`

`<footer>`Parte finale della pagina`</footer>`

Perché è utile:

- **rende il codice più leggibile;**
- aiuta accessibilità;
- aiuta motori di ricerca;
- rende la pagina più facile da mantenere;
- **evita una pagina fatta solo di `<div>` senza significato.**

MDN sottolinea che HTML, se scritto correttamente, dovrebbe definire anche la semantica del contenuto, utile per accessibilità e SEO.

HTML semantico

Esempio

```
<body>
  <header>
    <h1>Guida Git Interattiva</h1>
    <p>Documentazione semplice per imparare Git.</p>
  </header>

  <nav>
    <a href="#funzionalita">Funzionalità</a>
    <a href="#guida">Guida rapida</a>
    <a href="#documentazione">Documentazione</a>
  </nav>

  <main>
    <section id="funzionalita">
      <h2>Funzionalità</h2>
      <p>La pagina raccoglie comandi Git, esempi ed errori comuni.</p>
    </section>
  </main>

  <footer>
    <p>Progetto FISM 2026</p>
  </footer>
</body>
```

Tabelle HTML

Le tabelle servono per dati tabellari.

Non devono essere usate per impaginare tutta la pagina.

```
<table>
  <thead>
    <tr>
      <th>Comando</th>
      <th>Descrizione</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>git status</td>
      <td>Mostra lo stato della repository</td>
    </tr>
    <tr>
      <td>git add</td>
      <td>Aggiunge modifiche allo staging</td>
    </tr>
  </tbody>
</table>
```

Form base

Un form permette all'utente di inserire dati.

Elementi principali:

- `<form>`: contenitore del modulo;
- `<label>`: descrizione del campo;
- `<input>`: campo di input;
- `<button>`: pulsante.

```
<form>
  <label for="search">Cerca nella documentazione</label>
  <input type="text" id="search" name="search">
  <button type="submit">Cerca</button>
</form>
```

Il tag `<form>` rappresenta una sezione con controlli interattivi per inviare informazioni.

Oggi il form non deve per forza funzionare davvero.

Serve a costruire la struttura.

Lo faremo funzionare con JavaScript in una lezione successiva.

Esempio - Form Base

Aggiungere un form "Richiedi informazioni".

```
<section id="contatti">
  <h2>Richiedi informazioni</h2>

  <form>
    <label for="nome">Nome</label>
    <input type="text" id="nome" name="nome">

    <label for="messaggio">Messaggio</label>
    <textarea id="messaggio" name="messaggio"></textarea>

    <button type="submit">Invia</button>
  </form>
</section>
```



Menu di navigazione

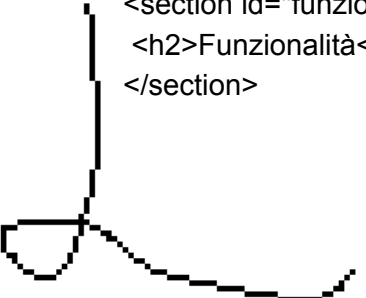
Il menu permette di muoversi tra sezioni della stessa pagina.

HTML:

```
<nav>
  <a href="#funzionalita">Funzionalità</a>
  <a href="#guida">Guida rapida</a>
  <a href="#documentazione">Documentazione</a>
  <a href="#screenshot">Screenshot</a>
</nav>
```

Le sezioni devono avere gli id corrispondenti:

```
<section id="funzionalita">
  <h2>Funzionalità</h2>
</section>
```



Esercizio lungo 1: creare index.html

Creare la prima homepage HTML

La pagina deve contenere:

- `header`;
- titolo progetto;
- descrizione breve;
- menu di navigazione;
- sezione funzionalità;
- sezione guida rapida;
- sezione documentazione;
- sezione struttura progetto;
- footer.

Vincoli

Usare almeno:

- `h1`;
- `h2`;
- `p`;
- `ul`;
- `ol`;
- `a`;
- `section`;
- `footer`.
- `table`



Esercizio lungo: creare index.html



Soluzione su la repo github del corso

<https://github.com/Duccioo/corso-FISM-2026>

Esercizio: conversione Markdown → HTML

Convertire questo contenuto Markdown in HTML semantico.

Catalogo Film

Catalogo Film è una piccola applicazione didattica per consultare una lista di film.

Funzionalità

- Ricerca per titolo
- Filtro per genere
- Visualizzazione dettagli

Guida rapida

1. Apri la pagina
2. Scrivi una parola nella barra di ricerca
3. Seleziona un film
4. Leggi i dettagli

Documentazione

- [Installazione](docs/installazione.md)
- [FAQ](docs/faq.md)

Esercizio: conversione

Convertire questo contenuto Markdown in HTML semantico.

```
<header>
<h1>Catalogo Film</h1>
<p>
  Catalogo Film è una piccola applicazione didattica
  per consultare una lista di film.
</p>
</header>

<main>
  <section id="funzionalita">
    <h2>Funzionalità</h2>
    <ul>
      <li>Ricerca per titolo</li>
      <li>Filtro per genere</li>
      <li>Visualizzazione dettagli</li>
    </ul>
  </section>

  <section id="guida">
    <h2>Guida rapida</h2>
    <ol>
      <li>Apri la pagina.</li>
      <li>Scrivi una parola nella barra di ricerca.</li>
      <li>Seleziona un film.</li>
      <li>Leggi i dettagli.</li>
    </ol>
  </section>

  <section id="documentazione">
    <h2>Documentazione</h2>
    <ul>
      <li><a href="docs/installazione.md">Installazione</a></li>
      <li><a href="docs/faq.md">FAQ</a></li>
    </ul>
  </section>
</main>
```

Cos'è CSS

CSS significa **Cascading Style Sheets**.

CSS serve a controllare la presentazione grafica della pagina:

- colori;
- font;
- dimensioni;
- margini;
- padding;
- bordi;
- spaziature;
- layout;
- comportamento responsive.

HTML risponde alla domanda:

Che cosa c'è nella pagina?

CSS risponde alla domanda:

Come deve apparire?

```
body {  
  font-family: Arial, sans-serif;  
  line-height: 1.6;  
}  
  
h1 {  
  color: #222;  
}
```

Collegare CSS a HTML

Per usare un file CSS esterno, bisogna collegarlo dentro `<head>`.

```
<head>
<meta charset="UTF-8">
<title>Nome progetto</title>
<link rel="stylesheet" href="style.css">
</head>
```

Struttura file:

```
nome-progetto/
├── index.html
└── style.css
```

Se il CSS non funziona, controllare:

- il file si chiama davvero `style.css`?
- è nella stessa cartella di `index.html`?
- il percorso in `href` è corretto?
- il browser ha ricaricato la pagina?
- ci sono errori di sintassi nel CSS?

Esercizio - collegare il CSS

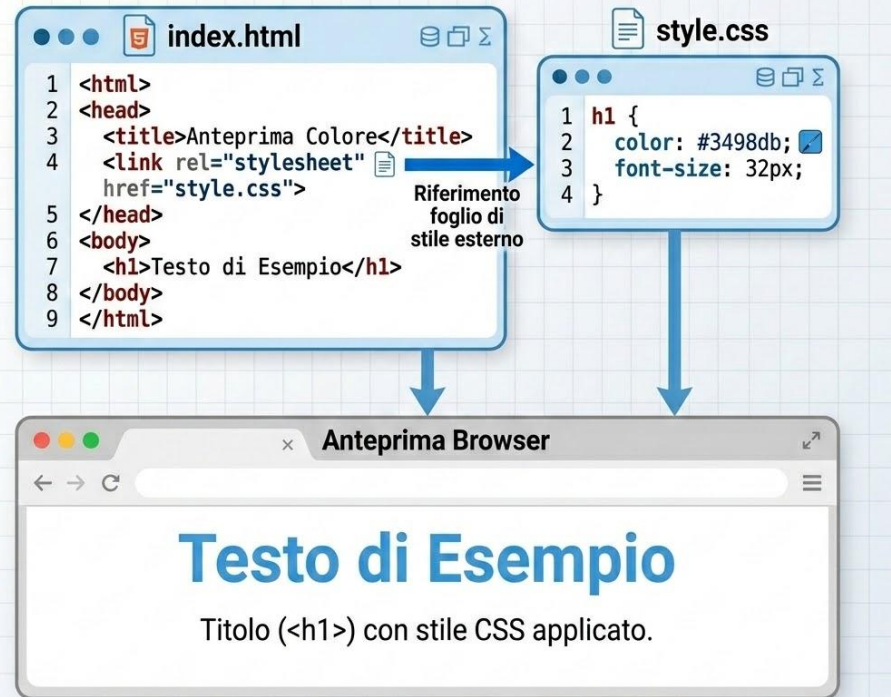
Collegare `style.css` alla pagina e cambiare il colore del titolo principale.

HTML:

```
<link rel="stylesheet" href="style.css">
```

CSS:

```
h1 {  
  color: #1f2937;  
}
```



Selettori CSS

Un selettore CSS indica quali elementi vogliamo modificare.

Selettore per tag:

```
p {  
  color: #333;  
}
```

Selettore per classe:

```
.card {  
  border: 1px solid #ddd;  
}
```

Selettore per id:

```
#funzionalita {  
  background: #f5f5f5;  
}
```

HTML COLLEGATO:

```
<section id="funzionalita" class="card">  
  <h2>Funzionalità</h2>  
  <p>Descrizione della sezione.</p>  
</section>
```

Regola pratica:

- *tag: stile generale;*
- *classi: componenti riutilizzabili;*
- *id: sezioni uniche o ancore.*

Esempio - Selettori CSS

Aggiungere la classe `card` a due sezioni e applicare bordo e padding.

```
.card {  
  border: 1px solid #ddd;  
  padding: 16px;  
  margin-bottom: 16px;  
}
```

```
<section class="card">  
  <h2>Funzionalità</h2>  
  <p>Elenco delle funzionalità principali.</p>  
</section>  
  
<section class="card">  
  <h2>Documentazione</h2>  
  <p>Link ai documenti principali.</p>  
</section>
```


Colori, font e leggibilità

Una pagina deve prima di tutto essere leggibile.

CSS base consigliato:

```
body {  
  font-family: Arial,  
  sans-serif;  
  color: #222;  
  background: #f7f7f7;  
  line-height: 1.6;  
}
```

Titoli:

```
h1, h2 {  
  color: #111827;  
}
```

Link:

```
a {  
  color: #0366d6;  
}  
  
a:hover {  
  text-decoration: underline;  
}
```

Regole pratiche:

- evitare troppi colori;
- evitare font strani;
- usare contrasto sufficiente;
- non riempire la pagina di effetti;
- lasciare spazio tra le sezioni.

Box model

In CSS ogni elemento può essere pensato come una scatola.

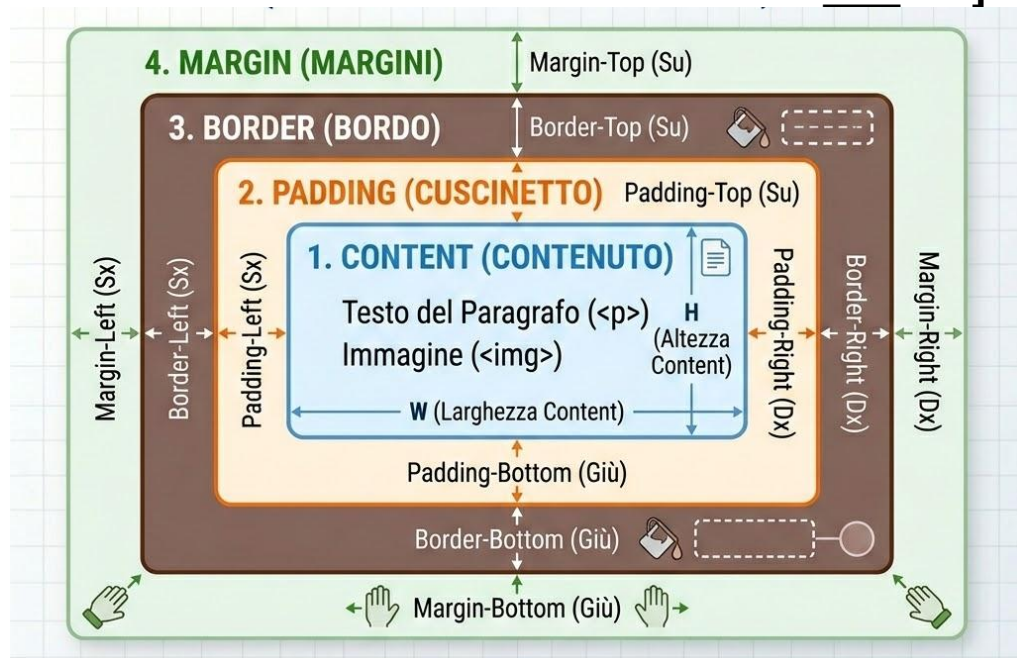
La scatola ha:

- contenuto;
- padding;
- bordo;
- margine.

```
.card {  
  padding: 16px;  
  border: 1px solid #ddd;  
  margin-bottom: 20px;  
}
```

Significato:

- **padding**: spazio interno;
- **border**: bordo;
- **margin**: spazio esterno.



Card layout

Le card sono blocchi visivi utili per separare informazioni.

Flexbox è pensato per disporre elementi in una dimensione, in riga o colonna, distribuendo lo spazio disponibile (`display: flex`, `gap`, `flex-wrap`).

```
<div class="cards">
  <section class="card">
    <h3>README</h3>
    <p>Panoramica del progetto.</p>
  </section>

  <section class="card">
    <h3>Guida utente</h3>
    <p>Istruzioni per usare il progetto.</p>
  </section>

  <section class="card">
    <h3>FAQ</h3>
    <p>Risposte alle domande frequenti.</p>
  </section>
</div>
```

```
.cards {
  display: flex;
  gap: 16px;
  flex-wrap: wrap;
}

.card {
  flex: 1;
  min-width: 220px;
  background: white;
  padding: 20px;
  border: 1px solid #e5e7eb;
  border-radius: 8px;
}
```



Esercizio lungo 2: rendere la pagina leggibile

Applicare CSS alla pagina dell'esercizio lungo 1.

La pagina deve avere:

- header scuro o evidenziato;
- contenuto centrale con larghezza massima;
- sezioni separate;
- almeno tre card;
- menu leggibile;
- footer;
- tabella leggibile;
- immagine ridimensionata correttamente.

Requisiti CSS

Usare:

- `font-family;`
- `background;`
- `color;`
- `margin;`
- `padding;`
- `border;`
- `border-radius;`
- `max-width;`
- `display: flex;`
- `gap.`



Esercizi Extra 1.

Sistemare una pagina HTML rotta

```
<html>
```

```
<head>
```

```
<title>Progetto
```

```
<body>
```

```
<h1>Catalogo Film
```

```
<p>Questo progetto permette di cercare film
```

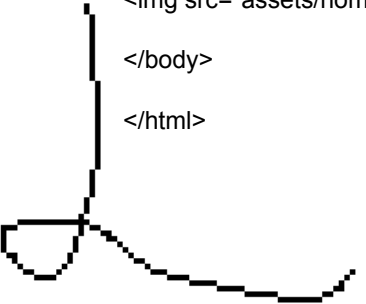
```
<a>Vai alla documentazione</a>
```

```

```

```
</body>
```

```
</html>
```





Esercizi Extra 2.

Creare una sezione FAQ in HTML

Esercizi Extra 2.

Creare una sezione FAQ in HTML

```
<section id="faq">
  <h2>FAQ</h2>

  <article class="card">
    <h3>Devo installare qualcosa?</h3>
    <p>No, il progetto funziona aprendo il file <code>index.html</code> nel browser.</p>
  </article>

  <article class="card">
    <h3>Perché il CSS non si vede?</h3>
    <p>Probabilmente il file <code>style.css</code> non è collegato correttamente.</p>
  </article>
</section>
```

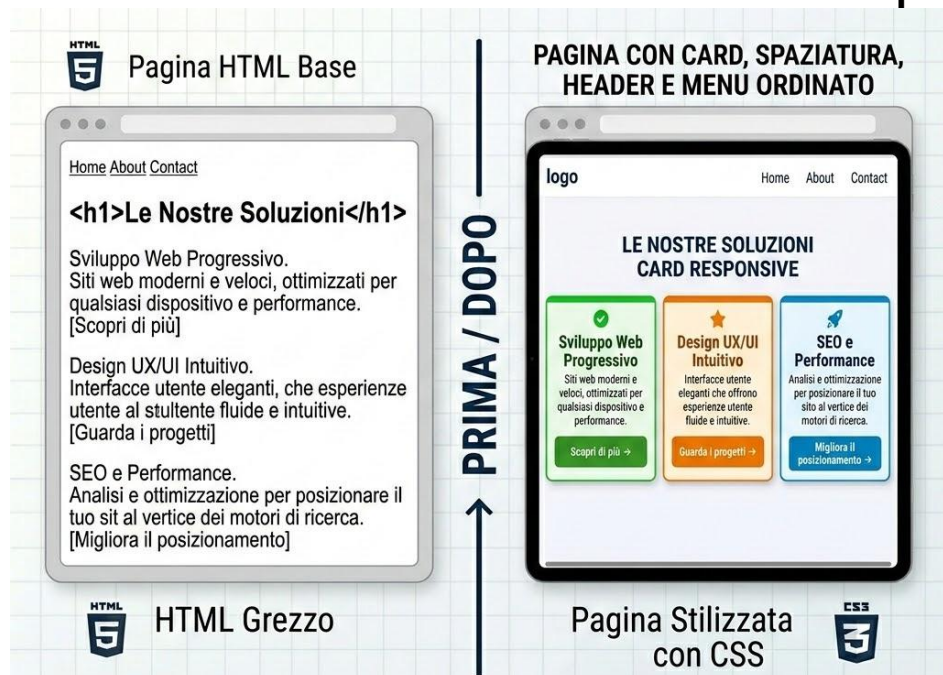
Oggi il CSS non serve solo a colorare

CSS non serve solo a “mettere i colori”.

CSS serve a controllare:

- leggibilità;
- spaziatura;
- disposizione degli elementi;
- dimensione del contenuto;
- comportamento su schermi diversi;
- aspetto di pulsanti, card, menu e tabelle;
- coerenza visiva della pagina;
- accessibilità minima.

Un sito può avere HTML corretto ma risultare comunque difficile da usare se il CSS è scritto male.



Come il browser costruisce una pagina

Quando apriamo `index.html`, il browser non mostra “magicamente” la pagina.

Il browser:

1. legge il file HTML;
2. costruisce la struttura della pagina;
3. trova i collegamenti a CSS, immagini e altri file;
4. carica `style.css`;
5. applica le regole CSS agli elementi HTML;
6. mostra il risultato finale.

Struttura tipica:

progetto/

```
|— index.html
|— style.css
|— assets/
|   |— screenshots/
|       |— homepage.png
|— docs/
    |— guida-utente.md
```

DevTools: lo strumento per non andare a caso

Quando qualcosa non funziona, non bisogna indovinare.

Bisogna usare gli strumenti del browser.

Con DevTools possiamo:

- ispezionare elementi HTML;
- vedere quali regole CSS sono applicate;
- modificare CSS temporaneamente;
- capire se una regola viene sovrascritta;
- controllare errori nella console;
- vedere se un file CSS o un'immagine non viene caricata;
- simulare uno schermo mobile.

Come aprirli:

- tasto destro sulla pagina;
- "Ispeziona";
- oppure **F12**;
- oppure **Ctrl + Shift + I**.



Esercizio - DevTools

1. Aprire `index.html` nel browser.
2. Fare tasto destro sul titolo.
3. Selezionare “Ispeziona”.
4. Cambiare il colore del titolo da DevTools.
5. Copiare la modifica vera dentro `style.css`.

Block, inline e flusso della pagina

Gli elementi HTML non si comportano tutti nello stesso modo.

Alcuni elementi sono **block**:

- occupano tutta la larghezza disponibile;
- iniziano su una nuova riga;
- possono contenere altri blocchi.

```
<h1>Titolo</h1>
<p>Paragrafo</p>
<section>Sezione</section>
<div>Contenitore generico</div>
```

Altri elementi sono **inline**:

- occupano solo lo spazio necessario;
- restano nella stessa riga;
- sono pensati per frammenti di contenuto.

```
<a href="#">Link</a>
<span>Testo evidenziato</span>
<strong>Testo importante</strong>
<code>git status</code>
```

Esempio pratico:

```
<p>Primo paragrafo</p>
<p>Secondo paragrafo</p>
```

```
<a href="#">Link uno</a>
<a href="#">Link due</a>
```

I paragrafi vanno uno sotto l'altro.

I link restano sulla stessa riga.

div e span: contenitori generici

`div` e `span` sono contenitori generici.

Non hanno un significato specifico.

Si usano quando non esiste un tag semantico più adatto.

`div` è un contenitore block:

```
<div class="card">  
  <h2>FAQ</h2>  
  <p>Risposte alle domande frequenti.</p>  
</div>
```

`span` è un contenitore inline:

```
<p>  
  Stato progetto:  
  <span class="badge">in sviluppo</span>  
</p>
```

Regola pratica:

Prima scegliere un tag semantico:

```
<header>  
<nav>  
<main>  
<section>  
<footer>
```

Solo dopo usare `div` e `span`.

display: cambiare il comportamento degli elementi

La proprietà `display` controlla come un elemento viene mostrato.

Valori base:

`display: block;`

`display: inline;`

`display: inline-block;`

`display: none;`

`display: flex;`

`display: grid;`

display: cambiare il comportamento degli elementi

Esempio:

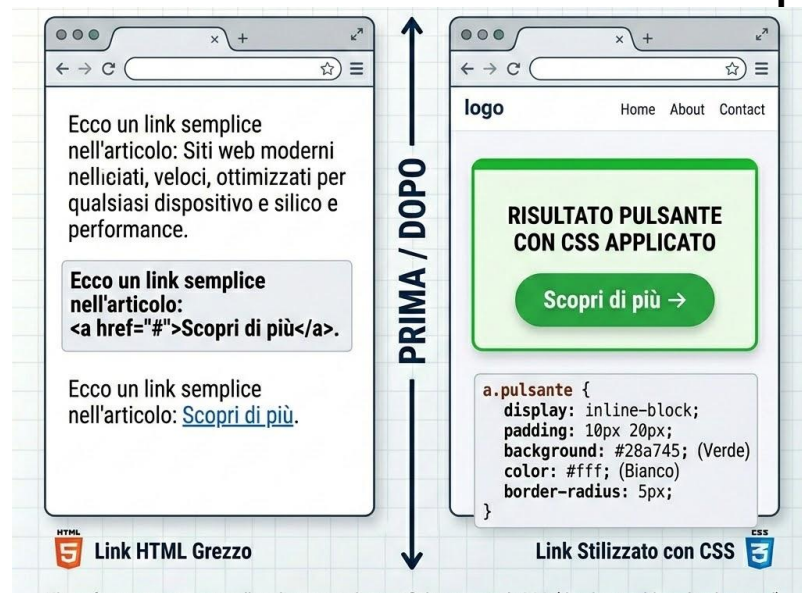
```
<a class="button" href="docs/guida-utente.md">Leggi la guida</a>
```

Un link è inline.

Se vogliamo farlo sembrare un bottone:

```
.button {  
  display: inline-block;  
  padding: 10px 16px;  
  background: #2563eb;  
  color: white;  
  text-decoration: none;  
  border-radius: 8px;  
}
```

inline-block mantiene l'elemento nella riga, ma permette di applicare padding, larghezza e altezza in modo più prevedibile.



CSS: cascata, ereditarietà e conflitti

CSS significa **Cascading Style Sheets**.

“Cascading” vuol dire che le regole possono sovrapporsi.

Quando più regole modificano lo stesso elemento, il browser decide quale vince.

Esempio:

```
<p class="intro">Questo è un testo introduttivo.</p>
```

```
p {  
  color: blue;  
}
```

```
.intro {  
  color: red;  
}
```

Il testo diventa rosso. Perché? Perché **.intro** è più specifico di **p**.

Altro esempio:

```
.intro {  
  color: red;  
}
```

```
.intro {  
  color: green;  
}
```

Il testo diventa verde.

Perché?

Perché a parità di specificità vince la regola scritta dopo.

CSS: cascata, ereditarietà e conflitti

Selettore	Esempio	Forza
Tag	<code>p</code>	bassa
Classe	<code>.intro</code>	media
ID	<code>#titolo</code>	alta
Inline style	<code>style="color:red"</code>	molto alta

Unità di misura CSS

In CSS possiamo misurare dimensioni e spaziature in modi diversi.

Unità	Significato	Uso tipico
px	pixel, misura fissa	bordi, piccoli dettagli
%	percentuale del contenitore	larghezze flessibili
rem	rispetto alla dimensione base del testo	font, spaziature coerenti
vh	percentuale dell'altezza dello schermo	sezioni grandi
vw	percentuale della larghezza dello schermo	casi particolari

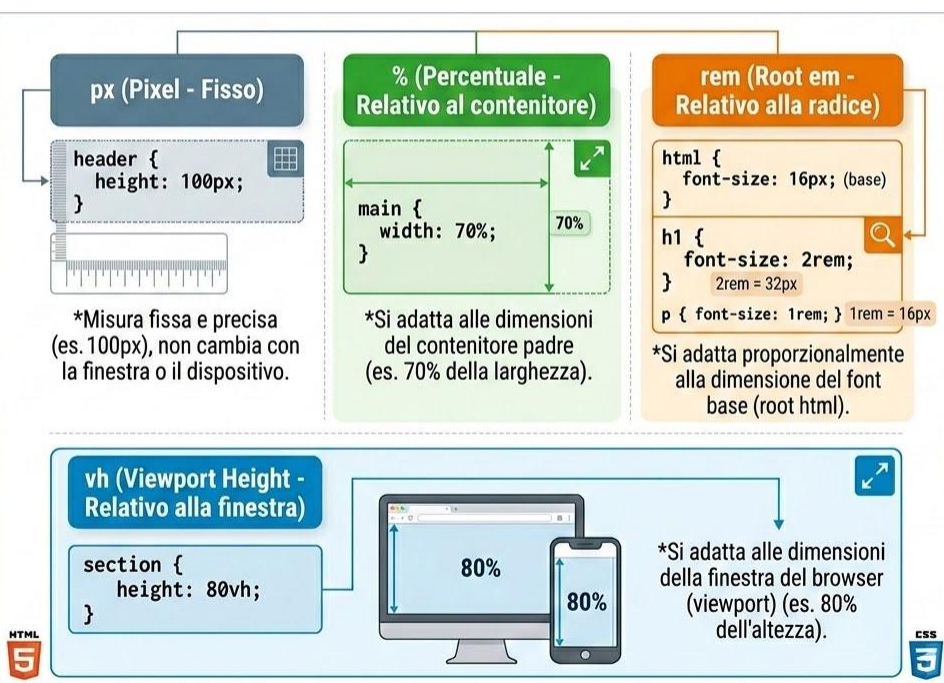
Unità di misura CSS

ESEMPIO:

```
.container {  
  width: 90%;  
  max-width: 1000px;  
  margin: 0 auto;  
}  
  
h1 {  
  font-size: 2.5rem;  
}  
  
.hero {  
  min-height: 60vh;  
}
```

Regola pratica:

- usare **px** per bordi e dettagli;
- usare **%** e **max-width** per contenitori;
- usare **rem** per testi e spaziature;
- usare **vh** solo per sezioni grandi.



Container e larghezza massima

Una pagina leggibile non dovrebbe occupare tutta la larghezza dello schermo.

Testo troppo largo = difficile da leggere.

Soluzione: usare un contenitore centrale.

```
<main class="container">
  <section>
    <h2>Descrizione</h2>
    <p>Testo del progetto.</p>
  </section>
</main>
```

```
.container {
  width: 90%;
  max-width: 1000px;
  margin: 32px auto;
}
```

width: 90%: il contenitore occupa il 90% dello spazio disponibile;
max-width: 1000px: oltre 1000px non cresce;
margin: 32px auto: margine verticale e centratura orizzontale.

Containe

Una pagina leggibile
schermo.

Testo troppo largo = c

Soluzione: usare un c

PANNELLO 1: TESTO TROPPO LARGO (No Container)



*Testo che occupa l'intera
larghezza dello schermo,
riducendo la leggibilità.*

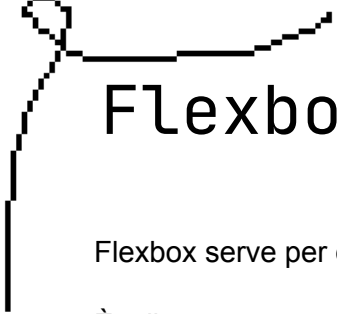
PANNELLO 2: CONTAINER LEGGIBILE (Centrato)



*Container centrale a larghezza
limitata (es. 800px) per una
migliore leggibilità.*



`width: 90%`: il contenitore occupa il 90% dello spazio disponibile;
`max-width: 1000px`: oltre 1000px non cresce;
`margin: 32px auto`: margine verticale e centratura orizzontale.



Flexbox: layout in una direzione



Flexbox serve per disporre elementi lungo una direzione principale.

È utile per:

- menu;
 - card affiancate;
 - bottoni;
 - sezioni con immagine e testo;
 - allineamento verticale/orizzontale.
- 

Flexbox: layout in una direzione

Flexbox serve per disporre elementi lungo una direzione principale.

Significato:

- **display: flex:** attiva Flexbox;
- **gap:** spazio tra gli elementi;
- **flex-wrap: wrap:** permette agli elementi di andare a capo;
- **flex: 1:** distribuisce lo spazio;
- **min-width:** evita card troppo strette.

```
.cards {  
  display: flex;  
  gap: 20px;  
  flex-wrap: wrap;  
}  
  
.card {  
  flex: 1;  
  min-width: 240px;  
}
```

```
<div class="cards">  
  <section class="card">  
    <h3>README</h3>  
    <p>Presenta il progetto.</p>  
  </section>  
  
  <section class="card">  
    <h3>Guida utente</h3>  
    <p>Spiega come usare il sito.</p>  
  </section>  
  
  <section class="card">  
    <h3>FAQ</h3>  
    <p>Risponde alle domande frequenti.</p>  
  </section>  
</div>
```

Flexbox: allineare un menu

Flexbox è molto utile per creare una navbar.

Proprietà principali:

- **justify-content**: allineamento sull'asse principale;
- **align-items**: allineamento sull'asse secondario;
- **gap**: spazio tra elementi;
- **flex-wrap**: permette di andare a capo.

```
<nav class="navbar">
  <a href="#descrizione">Descrizione</a>
  <a href="#funzionalita">Funzionalità</a>
  <a href="#guida">Guida rapida</a>
  <a href="#documentazione">Documentazione</a>
</nav>
```

```
.navbar {
  display: flex;
  justify-content: center;
  gap: 16px;
  padding: 16px;
  background: #f3f4f6;
}

.navbar a {
  color: #111827;
  text-decoration: none;
  font-weight: 600;
}

.navbar a:hover {
  text-decoration: underline;
}
```


CSS Grid: griglie semplici

CSS Grid serve per creare griglie bidimensionali.

Flexbox è ottimo per disporre elementi in una direzione.

Grid è ottimo per creare griglie di card o layout più strutturati.

Significato:

- `display: grid`: attiva Grid;
- `grid-template-columns`: definisce le colonne;
- `repeat(2, 1fr)`: due colonne uguali;
- `gap`: spazio tra righe e colonne.

HTML:

```
<div class="feature-grid">  
  <article class="card">Ricerca</article>  
  <article class="card">Filtro</article>  
  <article class="card">Dettagli</article>  
  <article class="card">Preferiti</article>  
</div>
```

CSS:

```
.feature-grid {  
  display: grid;  
  grid-template-columns: repeat(2, 1fr);  
  gap: 20px;  
}
```

CSS Grid: griglie semplici

CSS Grid serve per creare griglie bidimensionali.

Flexbox è ottimo per disporre elementi in una direzione.

Grid è ottimo per creare griglie di card o layout più strutturati.

Significato:

- `display: grid`: attiva Grid;
- `grid-template-columns`: definisce le colonne;
- `repeat(2, 1fr)`: due colonne uguali;
- `gap`: spazio tra righe e colonne.

HTML:

```
<div class="feature-grid">
  <article class="card">Ricerca</article>
  <article class="card">Filtro</article>
  <article class="card">Dettagli</article>
  <article class="card">Preferiti</article>
</div>
```

Versione più flessibile:

```
.feature-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 20px;
}
```

Flexbox o Grid?

Regola pratica:

- Flexbox: allineamento in una direzione.
- Grid: layout a righe e colonne.

Caso	Strumento consigliato
Menu orizzontale	Flexbox
Bottoni allineati	Flexbox
Card in una riga	Flexbox
Griglia di card	Grid
Layout a colonne	Grid
Sezione testo + immagine	Flexbox o Grid
Pagina intera complessa	Grid

Responsive design

Una pagina deve funzionare su schermi diversi:

- computer;
- tablet;
- smartphone;
- finestre ridimensionate;
- proiettore.

Errore tipico:

“Sul mio computer si vede bene, quindi va bene.”

No.

Bisogna controllare anche schermi piccoli.

Nel `<head>` bisogna inserire:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Questa riga dice al browser mobile di usare la larghezza reale del dispositivo.

Senza questa riga, il sito può apparire rimpicciolito o scomodo su smartphone.

Media query

Le media query permettono di cambiare CSS in base alla dimensione dello schermo.

```
@media (max-width: 700px) {  
  .navbar {  
    flex-direction: column;  
    align-items: center;  
  }  
  
  .hero {  
    text-align: center;  
  }  
  
  .cards {  
    grid-template-columns: 1fr;  
  }  
}
```

Significato:

Quando lo schermo è
largo al massimo 700px,
applica queste regole.

Media query

Le media query permettono di cambiare CSS in base alla dimensione dello schermo.

Prima:

```
.cards {  
  display: grid;  
  grid-template-columns: repeat(3,  
  1fr);  
}
```

Su mobile:

```
@media (max-width: 700px) {  
  .cards {  
    grid-template-columns: 1fr;  
  }  
}
```

Così le card passano da tre colonne a una colonna.

Immagini responsive

Le immagini troppo grandi rompono il layout.

È una delle regole CSS più utili e più semplici.

Regola base:

```
img {  
  max-width: 100%;  
  height: auto;  
}
```

Significato:

- l'immagine non supera la larghezza del contenitore;
- l'altezza resta proporzionata;
- la pagina non crea scroll orizzontale inutile.

Esempio HTML:

```

```

CSS:

```
.screenshot {  
  max-width: 100%;  
  height: auto;  
  border: 1px solid #e5e7eb;  
  border-radius: 12px;  
}
```

Stati interattivi: hover e focus

CSS può modificare l'aspetto degli elementi in base allo stato.

Esempio con **hover**:

```
.button:hover {  
  background: #1d4ed8;  
}
```

Esempio con **focus**:

```
input:focus {  
  outline: 2px solid #2563eb;  
  outline-offset: 2px;  
}
```

Stato	Significato
<code>:hover</code>	mouse sopra l'elemento
<code>:focus</code>	elemento selezionato da tastiera/click
<code>:active</code>	elemento mentre viene premuto

Organizzare style.css

Un file CSS disordinato diventa difficile da modificare.

Conviene dividerlo in sezioni commentate.

Ordine consigliato:

1. reset/base;
2. tipografia;
3. layout;
4. componenti;
5. tabelle/form;
6. responsive.

```
/* =====  
Base  
===== */  
  
* {  
  box-sizing: border-box;  
}  
  
body {  
  margin: 0;  
  font-family: Arial, sans-serif;  
  line-height: 1.6;  
}  
  
/* =====  
Layout  
===== */  
  
.container {  
  width: 90%;  
  max-width: 1000px;  
  margin: 32px auto;  
}  
  
/* =====  
Componenti  
===== */  
  
.card {  
  background: white;  
  padding: 24px;  
  border: 1px solid #e5e7eb;  
  border-radius: 12px;  
}  
  
/* =====  
Responsive  
===== */  
  
@media (max-width: 700px) {  
  .navbar {  
    flex-direction: column;  
  }  
}
```

Mini design system del progetto

Anche un piccolo progetto può avere regole visive coerenti.

Un mini design system definisce:

- colori principali;
- font;
- spaziature;
- bottoni;
- card;
- alert;
- badge;
- tabelle.

Vantaggio:

- Se voglio cambiare colore principale, lo cambio in un punto solo.

Esempio con variabili CSS:

```
:root {  
  --color-primary: #2563eb;  
  --color-dark: #111827;  
  --color-light: #f8fafc;  
  --color-border: #e5e7eb;  
  --radius: 12px;  
  --space: 16px;  
}
```

Uso:

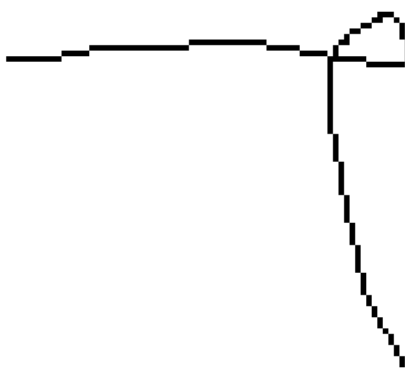
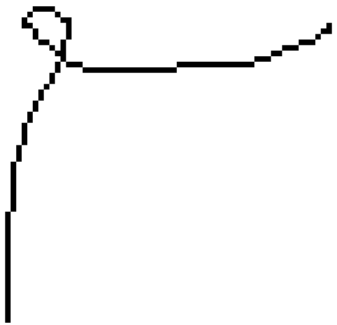
```
.button {  
  background: var(--color-primary);  
  border-radius: var(--radius);  
  padding: 10px 16px;  
}  
  
.card {  
  border: 1px solid  
  var(--color-border);  
  border-radius: var(--radius);  
  padding: calc(var(--space) * 1.5);  
}
```

Refactoring CSS: da brutto a ordinato

```
h1 {  
  color: blue;  
}  
  
.box {  
  padding: 20px;  
  border: 1px solid black;  
}  
  
.box2 {  
  padding: 20px;  
  border: 1px solid black;  
}  
  
.bottone {  
  background: blue;  
  color: white;  
}
```



```
:root {  
  --color-primary: #2563eb;  
  --color-border: #e5e7eb;  
  --radius: 12px;  
}  
  
.section-title {  
  color: var(--color-primary);  
}  
  
.card {  
  padding: 20px;  
  border: 1px solid var(--color-border);  
  border-radius: var(--radius);  
}  
  
.button {  
  display: inline-block;  
  background: var(--color-primary);  
  color: white;  
  padding: 10px 16px;  
  border-radius: var(--radius);  
  text-decoration: none;  
}
```



FINE



— WWW.DRACO.ME —

